UD 4. Gestión de eventos y formularios

1. Eventos en JSX

En React, los eventos se gestionan de manera diferente al HTML tradicional. JSX utiliza un sistema propio que se basa en atributos camelCase y en la asignación de funciones en lugar de cadenas de texto.

1.1 *Características principales*

Sintaxis camelCase

- En HTML: onclick="función()"
- En React: onClick={función}

Asignación de funciones, no cadenas

- React no interpreta cadenas como código. Se pasa una referencia a la función o una función anónima.

Eventos sintéticos

- React implementa un sistema llamado SyntheticEvent, que es una capa sobre los eventos nativos del navegador para asegurar compatibilidad entre navegadores.
- Ejemplo: e.preventDefault() funciona igual que en eventos nativos.

1.2. Eventos más comunes en React

A. **onClick** → clic en botones, enlaces, etc.
B. **onChange** → cambios en inputs, selects, textareas.
C. **onSubmit** → envío de formularios.
D. **onMouseEnter / onMouseLeave** → interacción con el ratón.
E. **onKeyDown / onKeyUp** → interacción con el teclado.

Ejemplo básico

/src/App.js

```
// src/App.js

import React, { useState } from 'react';

function App() {
  // Estado para inputs controlados
  const [texto, setTexto] = useState('');
  const [email, setEmail] = useState('');
  const [contadorClick, setContadorClick] = useState(0);
  const [hover, setHover] = useState(false);
  const [posicion, setPosicion] = useState({ x: 0, y: 0 });
  const [tecla, setTecla] = useState('');
  const [wheelDelta, setWheelDelta] = useState(0);
  const [touch, setTouch] = useState({ x: null, y: null });

  // Clicks, doble clic, menú contextual
  const manejarClick = () => {
    setContadorClick(c => c + 1);
  };
  const manejarDobleClick = () => {
    alert('Doble clic detectado');
  };
  const manejarContextMenu = (e) => {
    e.preventDefault();
    alert('Menú contextual bloqueado');
  };

  // Change, Input (controlados)
  const manejarCambioTexto = (e) => {
    setTexto(e.target.value);
  };
  const manejarInputEmail = (e) => {
    setEmail(e.target.value);
  };

  // Focus / Blur
  const manejarFocus = () => {
    console.log('Input obtuvo foco');
  };
  const manejarBlur = () => {
    console.log('Input perdió el foco');
  };

  // Keyboard
  const manejarKeyDown = (e) => {
```

```
    setTecla(`KeyDown: ${e.key}`);
    if (e.key === 'Enter') {
      e.preventDefault();
      alert('Enter presionado');
    }
  };
  const manejarKeyUp = (e) => {
    setTecla(`KeyUp: ${e.key}`);
  };

  // Mouse
  const manejarMouseEnter = () => setHover(true);
  const manejarMouseLeave = () => setHover(false);
  const manejarMouseMove = (e) => {
    setPosicion({ x: e.nativeEvent.offsetX, y:
e.nativeEvent.offsetY });
  };

  // Wheel (scroll con rueda)
  const manejarWheel = (e) => {
    setWheelDelta(e.deltaY);
  };

  // Touch (móvil)
  const manejarTouchStart = (e) => {
    const t = e.touches[0];
    setTouch({ x: t.clientX, y: t.clientY });
  };
  const manejarTouchMove = (e) => {
    const t = e.touches[0];
    setTouch({ x: t.clientX, y: t.clientY });
  };
  const manejarTouchEnd = () => {
    setTouch({ x: null, y: null });
  };

  // Submit formulario
  const manejarSubmit = (e) => {
    e.preventDefault();
    if (!email.trim()) {
      alert('El email es obligatorio');
      return;
    }
    alert(`Formulario enviado: texto=\"${texto}\",
email=\"${email}\"`);
  };

  return (
```

```
    <div style={{ fontFamily: 'system-ui, sans-serif', padding: 24,
lineHeight: 1.5 }}>
      <h1>Eventos comunes en React</h1>

      {/* Clicks */}
      <section style={{ marginBottom: 24 }}>
        <h2>Clicks</h2>
        <button onClick={manejarClick}
onDoubleClick={manejarDobleClick}
onContextMenu={manejarContextMenu}>
          Haz clic / doble clic / botón derecho
        </button>
        <p>Has hecho clic <strong>{contadorClick}</strong>
veces.</p>
      </section>

      {/* Formularios: change, input, submit, focus, blur */}
      <section style={{ marginBottom: 24 }}>
        <h2>Formularios</h2>
        <form onSubmit={manejarSubmit}>
          <div style={{ marginBottom: 12 }}>
            <label>
              Texto (onChange):
              <input
                type="text"
                value={texto}
                onChange={manejarCambioTexto}
                onFocus={manejarFocus}
                onBlur={manejarBlur}
                placeholder="Escribe algo"
                style={{ marginLeft: 8 }}
              />
            </label>
          </div>
          <div style={{ marginBottom: 12 }}>
            <label>
              Email (onInput):
              <input
                type="email"
                value={email}
                onInput={manejarInputEmail}
                placeholder="tu@correo.com"
                style={{ marginLeft: 8 }}
              />
            </label>
          </div>
          <button type="submit">Enviar</button>
        </form>
```

```
      </section>

      {/* Teclado */}
      <section style={{ marginBottom: 24 }}>
        <h2>Teclado</h2>
        <input
          type="text"
          onKeyDown={manejarKeyDown}
          onKeyUp={manejarKeyUp}
          placeholder="Pulsa alguna tecla"
        />
        <p>{tecla}</p>
      </section>

      {/* Ratón */}
      <section style={{ marginBottom: 24 }}>
        <h2>Ratón</h2>
        <div
          onMouseEnter={manejarMouseEnter}
          onMouseLeave={manejarMouseLeave}
          onMouseMove={manejarMouseMove}
          style={{
            width: 300,
            height: 160,
            border: '2px dashed #888',
            borderRadius: 8,
            display: 'flex',
            alignItems: 'center',
            justifyContent: 'center',
            background: hover ? '#f0fff4' : '#f9fafb',
          }}
        >
          <div>
            <div>Área interactiva (move/enter/leave)</div>
            <div style={{ fontSize: 12, color: '#555', marginTop: 8
}}>
              Posición: x={posicion.x} y={posicion.y}
            </div>
          </div>
        </div>
      </section>

      {/* Wheel */}
      <section style={{ marginBottom: 24 }}>
        <h2>Rueda del ratón (Wheel)</h2>
        <div
          onWheel={manejarWheel}
          style={{ width: 300, height: 100, overflow: 'auto',
```

```
border: '1px solid #ccc', padding: 12 }}
        >
          <p>Haz scroll con la rueda del ratón sobre este
recuadro.</p>
          <p style={{ fontSize: 12, color: '#555' }}>deltaY:
{wheelDelta}</p>
          <div style={{ height: 300 }} />
        </div>
      </section>

      {/* Touch */}
      <section style={{ marginBottom: 24 }}>
        <h2>Eventos táctiles (Touch)</h2>
        <div
          onTouchStart={manejarTouchStart}
          onTouchMove={manejarTouchMove}
          onTouchEnd={manejarTouchEnd}
          style={{
            width: 300,
            height: 120,
            border: '2px solid #888',
            borderRadius: 8,
            display: 'flex',
            alignItems: 'center',
            justifyContent: 'center',
            background: '#fffef0',
          }}
        >
          {touch.x === null ? (
            <span>Toca y arrastra en un dispositivo táctil</span>
          ) : (
            <span>Touch: x={touch.x} y={touch.y}</span>
          )}
        </div>
      </section>

      <footer style={{ marginTop: 32, fontSize: 12, color: '#666'
}}>
        <p>
          Nota: estos son los eventos más comunes. React también
soporta eventos para formularios avanzados, portales, y más.
Consulta la documentación oficial para detalles completos.
        </p>
      </footer>
    </div>
  );
}
```

```
export default App;
```

1.3. Pasar parámetros a eventos

Para pasar argumentos, se utiliza una función flecha:

Ejemplo:

/src/App.js

```
// src/App.js

import React, { useState } from 'react';

/**
 * App.js — Pasar parámetros a eventos en React (versión corregida)
 *
 * Se han eliminado fragmentos JSX con <code>...</code> que
incluían identificadores
 * como "handler" y "param" dentro de llaves, lo que podía provocar
advertencias
 * de ESLint (no-undef) si el parser los interpretaba como expresiones.
 *
 * En su lugar, los ejemplos se muestran como texto seguro usando
<pre> y strings.
 */

// === 1) Flecha inline ===
function InlineArrowDemo({ nombre }) {
  const saludar = (n) => alert(`Hola, ${n}`);

  const ejemplo = `onClick={() => handler(param)}`;

  return (
    <section style={{ marginBottom: 24 }}>
      <h2>1) Función flecha inline</h2>
      <button onClick={() => saludar(nombre)}>Saludar a
{nombre}</button>
      <pre style={{ background: '#f7f7f7', padding: 8, borderRadius: 4
}}>
        <code>{ejemplo}</code>
      </pre>
      <p style={{ fontSize: 12, color: '#666' }}>
        Uso directo de una función flecha en el atributo del evento para
pasar parámetros.
      </p>
    </section>
  );
```

```
}

// === 2) Curried handler ===
function CurriedDemo() {
  const [contador, setContador] = useState(0);
  const sumar = (cantidad) => (e) => {
    e.preventDefault();
    setContador((c) => c + cantidad);
  };

  const ejemplo = `const handler = (param) => (e) => { /* lógica */
}`;

  return (
    <section style={{ marginBottom: 24 }}>
      <h2>2) Manejador curried</h2>
      <button onClick={sumar(1)}>+1</button>
      <button onClick={sumar(5)} style={{ marginLeft: 8
}}>+5</button>
      <button onClick={sumar(-1)} style={{ marginLeft: 8 }}>-
1</button>
      <p>Contador: {contador}</p>
      <pre style={{ background: '#f7f7f7', padding: 8, borderRadius: 4
}}>
        <code>{ejemplo}</code>
      </pre>
    </section>
  );
}

// === 3) data-* attributes ===
function DataAttrDemo() {
  const manejarClick = (e) => {
    const id = e.currentTarget.dataset.id;
    const nombre = e.currentTarget.dataset.nombre;
    alert(`Clic en id=${id}, nombre=${nombre}`);
  };

  return (
    <section style={{ marginBottom: 24 }}>
      <h2>3) Atributos data-*</h2>
      <button data-id="42" data-nombre="Componente"
onClick={manejarClick}>
        Leer parámetros desde data-*
      </button>
      <p style={{ fontSize: 12, color: '#666' }}>
        Útil cuando el parámetro proviene del propio elemento DOM.
      </p>
```

```
    </section>
  );
}

// === 4) Múltiples parámetros + evento ===
function MultiParamsDemo() {
  const manejarAccion = (id, accion, e) => {
    e.stopPropagation();
    alert(`Acción: ${accion} sobre id=${id}`);
  };

  return (
    <section style={{ marginBottom: 24 }}>
      <h2>4) Múltiples parámetros + evento</h2>
      <div style={{ padding: 12, border: '1px solid #ccc' }} onClick={()
=> alert('Contenedor clicado')}>
        <button onClick={(e) => manejarAccion(7, 'editar', e)}>Editar
#7</button>
        <button onClick={(e) => manejarAccion(7, 'borrar', e)} style={{
marginLeft: 8 }}>Borrar #7</button>
      </div>
      <p style={{ fontSize: 12, color: '#666' }}>
        Pasa <code>(id, accion, e)</code> usando una flecha inline y
controla propagación.
      </p>
    </section>
  );
}

// === 5) Listas con parámetros ===
function ListaDemo() {
  const productos = [
    { id: 1, nombre: 'Teclado', precio: 25 },
    { id: 2, nombre: 'Ratón', precio: 15 },
    { id: 3, nombre: 'Pantalla', precio: 150 },
  ];

  const seleccionar = (id) => {
    alert(`Seleccionaste el producto #${id}`);
  };

  const verDetalle = (producto) => {
    alert(`Detalle → ${producto.nombre} (${producto.precio}€)`);
  };

  return (
    <section style={{ marginBottom: 24 }}>
      <h2>5) Listas: pasar id/índice/objeto</h2>
```

```
      <ul>
        {productos.map((p, idx) => (
          <li key={p.id} style={{ marginBottom: 8 }}>
            <strong>{idx + 1}.</strong> {p.nombre} — {p.precio}€
            <button onClick={() => seleccionar(p.id)} style={{
marginLeft: 8 }}>Seleccionar</button>
            <button onClick={() => verDetalle(p)} style={{ marginLeft: 8
}}>Ver detalle</button>
          </li>
        ))}
      </ul>
      <p style={{ fontSize: 12, color: '#666' }}>
        En listas se recomienda pasar el <code>id</code> o el objeto
completo según la necesidad.
      </p>
    </section>
  );
}

// === 6) Submit con parámetros externos ===
function SubmitDemo() {
  const [email, setEmail] = useState('');
  const origen = 'landing-campaña';

  const enviar = (canal) => (e) => {
    e.preventDefault();
    if (!email.trim()) {
      alert('Email obligatorio');
      return;
    }
    alert(`Submit → email=${email}, canal=${canal},
origen=${origen}`);
  };

  return (
    <section style={{ marginBottom: 24 }}>
      <h2>6) onSubmit con parámetros</h2>
      <form>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="tu@correo.com"
        />
        <div style={{ marginTop: 8 }}>
          <button onClick={enviar('web')}>Enviar por web</button>
          <button onClick={enviar('mobile')} style={{ marginLeft: 8
}}>Enviar por mobile</button>
```

```
        </div>
      </form>
      <p style={{ fontSize: 12, color: '#666' }}>
        El parámetro <code>canal</code> queda cerrado por la función
curried.
      </p>
    </section>
  );
}

export default function App() {
  return (
    <div style={{ fontFamily: 'system-ui, sans-serif', padding: 24 }}>
      <h1>Pasar parámetros a eventos en React</h1>
      <InlineArrowDemo nombre="Francisco" />
      <CurriedDemo />
      <DataAttrDemo />
      <MultiParamsDemo />
      <ListaDemo />
      <SubmitDemo />
      <footer style={{ marginTop: 32, fontSize: 12, color: '#666' }}>
        <p>
          Tip: evita crear funciones nuevas dentro del JSX en listas muy
grandes si impacta el rendimiento;
          puedes memorizar handlers con <code>useCallback</code>.
        </p>
      </footer>
    </div>
  );
}
```

2. Manejo de formularios: componentes controlados

2.1 Introducción a los formularios en React

Los formularios son una parte esencial de cualquier aplicación web, ya que permiten la interacción del usuario mediante la introducción y envío de datos. En React, el manejo de formularios se basa en el concepto de **componentes controlados**, lo que significa que el estado del formulario se gestiona dentro del componente, en lugar de depender del DOM como en HTML tradicional.

¿Por qué React maneja los formularios de forma diferente?

- En HTML, los elementos de formulario (inputs, selects, textareas) gestionan su propio estado.
- En React, se busca un **flujo de datos unidireccional**, por lo que el valor del input se sincroniza con el **estado del componente**.
- Esto permite:
  - Validación en tiempo real.
  - Control total sobre los datos.
  - Integración con lógica de negocio.

Componentes controlados

Un componente controlado es aquel en el que el valor del input está vinculado al estado del componente. Cada cambio en el input dispara un evento que actualiza el estado.

Ejemplo básico

/src/App.js

```
// src/App.js

import { useState } from "react";

function Formulario() {
  const [nombre, setNombre] = useState("");

  const manejarCambio = (e) => {
    setNombre(e.target.value);
  };

  const manejarSubmit = (e) => {
    e.preventDefault();
    alert(`Nombre enviado: ${nombre}`);
  };

  return (
    <form onSubmit={manejarSubmit}>
          <input      type="text"      value={nombre}
onChange={manejarCambio} />
      <button type="submit">Enviar</button>
    </form>
  );
}
```

Eventos clave en formularios

- onChange: Detecta cambios en inputs.
- onSubmit: Captura el envío del formulario.
- onFocus / onBlur: Detecta cuando un campo gana o pierde foco.
- onInput: Similar a onChange, pero se dispara en cada entrada.

Ventajas del enfoque controlado

- Validación sencilla antes de enviar.
- Posibilidad de mostrar mensajes de error dinámicos.
- Sincronización con otros componentes o lógica.

2.2 Acceso a elementos de formularios

En React, el acceso a los elementos de formularios se realiza principalmente mediante **componentes controlados**, donde el valor del input está vinculado al estado del componente. Esto permite un control total sobre los datos y facilita la validación.

A. Text

Los campos de texto son los más comunes. Se controlan con el evento onChange y el estado del componente.

B. Radio button

Los radio buttons permiten seleccionar **una opción entre varias**. Se controlan igual que los inputs, pero se agrupan por name.

C. Checkbox

Los checkboxes permiten seleccionar **una o varias opciones**. Se controlan con checked en lugar de value.

D. Select

El elemento <select> se controla con value y onChange.

Ejemplo completo

/src/App.js

```
//src/App.js
import React, { useState } from 'react';

// 2.1. Text (Inputs de texto)
function InputTexto() {
  const [nombre, setNombre] = useState('');
  const [error, setError] = useState('');

  const manejarCambio = (e) => {
    const v = e.target.value;
    setNombre(v);
    setError(v.trim().length < 3 ? 'Mínimo 3
caracteres' : '');
  };

  return (
    <section style={styles.section}>
      <h2>2.1. Input de texto</h2>
      <label style={styles.label}>
        Nombre:
        <input
          type="text"
          value={nombre}
          onChange={manejarCambio}
          placeholder="Escribe tu nombre"
          style={styles.input}
        />
      </label>
      <div style={styles.info}>Valor actual:
<strong>{nombre || '—'}</strong></div>
      {error && <div
style={styles.error}>{error}</div>}
    </section>
  );
}

// 2.2. Radio button (una opción entre varias)
function RadioDemo() {
  const [genero, setGenero] = useState('');

  const manejarCambio = (e) =>
setGenero(e.target.value);

  return (
    <section style={styles.section}>
```

```
      <h2>2.2. Radio button</h2>
      <p>Selecciona tu género:</p>
      <label style={styles.inlineLabel}>
        <input
          type="radio"
          name="genero"
          value="masculino"
          checked={genero === 'masculino'}
          onChange={manejarCambio}
        />
        Masculino
      </label>
      <label style={styles.inlineLabel}>
        <input
          type="radio"
          name="genero"
          value="femenino"
          checked={genero === 'femenino'}
          onChange={manejarCambio}
        />
        Femenino
      </label>
      <label style={styles.inlineLabel}>
        <input
          type="radio"
          name="genero"
          value="otro"
          checked={genero === 'otro'}
          onChange={manejarCambio}
        />
        Otro
      </label>
      <div style={styles.info}>Seleccionado:
<strong>{genero || '—'}</strong></div>
    </section>
  );
}

// 2.3. Checkbox (uno y múltiples)
function CheckboxSimple() {
  const [acepta, setAcepta] = useState(false);
  const manejarCambio = (e) =>
setAcepta(e.target.checked);

  return (
    <section style={styles.section}>
      <h2>2.3. Checkbox (simple)</h2>
      <label style={styles.inlineLabel}>
```

```
        <input type="checkbox" checked={acepta}
onChange={manejarCambio} />
        Acepto los términos
      </label>
      <div style={styles.info}>{acepta ? 'Aceptado'
: 'No aceptado'}</div>
    </section>
  );
}

function CheckboxLista() {
  const [intereses, setIntereses] = useState([]);
  const opciones = [
    { id: 'deporte', label: 'Deporte' },
    { id: 'musica', label: 'Música' },
    { id: 'cine', label: 'Cine' },
    { id: 'tecnologia', label: 'Tecnología' },
  ];

  const manejarCambio = (e) => {
    const valor = e.target.value;
    setIntereses((prev) =>
      prev.includes(valor) ? prev.filter((i) => i !==
valor) : [...prev, valor]
    );
  };

  return (
    <section style={styles.section}>
      <h3>2.3.b Checkboxes múltiples</h3>
      <p>Selecciona tus intereses:</p>
      <div>
        {opciones.map((op) => (
          <label key={op.id}
style={styles.inlineLabel}>
            <input
              type="checkbox"
              value={op.id}
              checked={intereses.includes(op.id)}
              onChange={manejarCambio}
            />
            {op.label}
          </label>
        ))}
      </div>
      <div style={styles.info}>Intereses:
<strong>{intereses.length ? intereses.join(', ') :
'—'}</strong></div>
```

```
    </section>
  );
}

// 2.4. Select (desplegable)
function SelectDemo() {
  const [pais, setPais] = useState('');
  const manejarCambio = (e) =>
setPais(e.target.value);

  return (
    <section style={styles.section}>
      <h2>2.4. Select</h2>
      <label style={styles.label}>
        País:
        <select value={pais}
onChange={manejarCambio}
style={styles.select}>
          <option value="">— Selecciona —
</option>
          <option value="es">España</option>
          <option value="mx">México</option>
          <option value="ar">Argentina</option>
          <option value="co">Colombia</option>
        </select>
      </label>
      <div style={styles.info}>Seleccionado:
<strong>{pais || '—'}</strong></div>
    </section>
  );
}

function SelectMultipleDemo() {
  const [lenguajes, setLenguajes] =
useState(['js']);
  const manejarCambio = (e) => {
    const seleccion =
Array.from(e.target.selectedOptions).map((o) =>
o.value);
    setLenguajes(seleccion);
  };

  return (
    <section style={styles.section}>
      <h3>2.4.b Select múltiple</h3>
      <label style={styles.label}>
        Lenguajes que usas:
        <select multiple value={lenguajes}
```

```
onChange={manejarCambio} style={{
...styles.select, height: 110 }}>
          <option value="js">JavaScript</option>
          <option value="ts">TypeScript</option>
          <option value="py">Python</option>
          <option value="java">Java</option>
        </select>
      </label>
      <div style={styles.info}>Seleccionados:
<strong>{lenguajes.length ? lenguajes.join(', ') :
'—'}</strong></div>
    </section>
  );
}

// Estilos simples inline para mejor legibilidad
const styles = {
  app: { fontFamily: 'system-ui, -apple-system,
Segoe UI, Roboto, sans-serif', padding: 24,
lineHeight: 1.5 },
  section: { marginBottom: 24, padding: 16,
border: '1px solid #e5e7eb', borderRadius: 8,
background: '#fff' },
  label: { display: 'block', marginBottom: 8 },
  inlineLabel: { display: 'inline-flex', alignItems:
'center', gap: 8, marginRight: 16 },
  input: { marginLeft: 8, padding: '6px 8px' },
  select: { marginLeft: 8, padding: '6px 8px' },
  info: { marginTop: 8, fontSize: 14, color:
'#374151' },
  error: { marginTop: 8, color: '#b91c1c',
fontWeight: 600 },
  header: { marginBottom: 16 },
};

export default function App() {
  return (
    <div style={styles.app}>
      <header style={styles.header}>
        <h1>2. Acceso a elementos de formularios
en React</h1>
        <p>Demostración de inputs controlados:
texto, radio, checkbox y select.</p>
      </header>

      <InputTexto />
      <RadioDemo />
      <CheckboxSimple />
```

```
      <CheckboxLista />
      <SelectDemo />
      <SelectMultipleDemo />

      <footer style={{ marginTop: 24, fontSize:
12, color: '#6b7280' }}>
        <p>
          Tip: todos los controles son
<strong>controlados</strong>: el estado vive en
React y el valor/checked lo reflejan.
        </p>
      </footer>
    </div>
  );
}
```

2.3 Validación de formularios: evento onsubmit

En React, el evento onSubmit es el punto central para validar y procesar un formulario antes de enviar datos. Al usar componentes **controlados** (estado en React), podemos:

- **Validar** todos los campos a la vez en el onSubmit.
- **Evitar dobles envíos** con un flag isSubmitting.
- **Enviar** el formulario **programáticamente** (desde código) con requestSubmit().

A. Validar un formulario

Objetivo: comprobar reglas de negocio cuando el usuario intenta enviar y evitar el envío si hay errores.
Clave técnica:

- En el onSubmit, llama a una función validate(values) que devuelve un objeto de errores por campo.
- Si hay errores, los muestras junto a cada input y **no continúas** con el envío.
- Usa noValidate en el <form> si quieres **desactivar los mensajes nativos** del navegador y tener control total.

Reglas típicas de ejemplo:

- nombre → obligatorio y con mínimo de caracteres.
- email → obligatorio y con formato válido (regex simple).

B. Deshabilitar enviar un formulario dos veces

Objetivo: evitar que el usuario haga doble clic o pulse Enter repetidamente mientras una petición está en curso.
Clave técnica:

- Usa un flag isSubmitting en estado.
- Si isSubmitting es true, **ignora** nuevos envíos y **deshabilita** el botón (y opcionalmente todo el <fieldset>).
- Tras la respuesta (o error), vuelve a false.

Extra:

- Un fieldset con disabled deshabilita **todos** los controles del formulario cómodamente.

C. Enviar un formulario desde código

Objetivo: disparar el envío de un formulario desde un botón externo o una acción de UI que no sea el botón submit.
Clave técnica:

- Obtén la referencia al formulario con useRef.
- Usa formRef.current.requestSubmit() para disparar el evento submit y **respetar validaciones**.
- Si tu entorno no soporta requestSubmit, como *fallback* puedes **lanzar** un evento submit nativo con dispatchEvent.

Nota: form.submit() **no** dispara los manejadores de submit (salta validación y eventos). Por eso es preferible requestSubmit().

Ejemplo completo

/src/App.js

```
// src/App.js

import React, { useRef, useState } from 'react';

/**
 * App.js — 3. Validación de formularios: evento onSubmit
 *
 * Incluye tres secciones:
 * 3.1. Validar un formulario (custom, en onSubmit)
 * 3.2. Deshabilitar enviar un formulario dos veces (isSubmitting + fieldset)
 * 3.3. Enviar un formulario desde código (requestSubmit con ref)
 *
```

```
 * Pensado para ejecutarse en Create React App
(CRA).
 */

// === 3.1 Validar un formulario ===
function ValidarFormulario() {
  const [values, setValues] = useState({ nombre:
'', email: '' });
  const [errors, setErrors] = useState({});
  const [submitted, setSubmitted] =
useState(false);

  const validate = (v) => {
    const e = {};
    if (!v.nombre.trim()) e.nombre = 'El nombre es
obligatorio';
    else if (v.nombre.trim().length < 3) e.nombre
= 'Mínimo 3 caracteres';

    if (!v.email.trim()) e.email = 'El email es
obligatorio';
    else if (!/^\S+@\S+\.\S+$/.test(v.email))
e.email = 'Formato de email inválido';

    return e;
  };

  const handleChange = (ev) => {
    const { name, value } = ev.target;
    setValues((prev) => ({ ...prev, [name]: value
}));
    // Limpia el error del campo cuando el usuario
edita
    if (errors[name]) setErrors((prev) => ({
...prev, [name]: undefined }));
  };

  const handleSubmit = (ev) => {
    ev.preventDefault();
    const e = validate(values);
    setErrors(e);
    const ok =
Object.values(e).filter(Boolean).length === 0;
    setSubmitted(ok);
    if (!ok) return;

    alert(`Formulario válido.
Nombre=\"${values.nombre}\",
```

```
Email=\"${values.email}\"`);
  };

  return (
    <section style={styles.section}>
      <h2>3.1. Validar un formulario (onSubmit)</h2>
      <form noValidate onSubmit={handleSubmit}>
        <div style={styles.fieldRow}>
          <label style={styles.label}>
            Nombre
            <input
              type="text"
              name="nombre"
              value={values.nombre}
              onChange={handleChange}
              aria-invalid={Boolean(errors.nombre)}
              style={{ ...styles.input, borderColor: errors.nombre ? '#dc2626' : '#d1d5db' }}
              placeholder="Tu nombre"
              required
            />
          </label>
          {errors.nombre && <div style={styles.error}>{errors.nombre}</div>}
        </div>

        <div style={styles.fieldRow}>
          <label style={styles.label}>
            Email
            <input
              type="email"
              name="email"
              value={values.email}
              onChange={handleChange}
              aria-invalid={Boolean(errors.email)}
              style={{ ...styles.input, borderColor: errors.email ? '#dc2626' : '#d1d5db' }}
              placeholder="tu@correo.com"
              required
            />
          </label>
          {errors.email && <div style={styles.error}>{errors.email}</div>}
        </div>

        <button type="submit"
```

```
style={styles.buttonPrimary}>Enviar</button>
      </form>

      <div style={styles.info}>Estado envío: {submitted ? 'Válido' : ' Pendiente/Inválido'}</div>
      <p style={styles.tip}>Tip: <code>noValidate</code> desactiva los mensajes nativos del navegador para usar validación custom.</p>
    </section>
  );
}

// === 3.2 Deshabilitar enviar dos veces ===
function DoubleSubmitDemo() {
  const [email, setEmail] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const fakeRequest = () => new Promise((resolve) => setTimeout(resolve, 1200));

  const handleSubmit = async (ev) => {
    ev.preventDefault();
    if (isSubmitting) return; // Guardia para doble clic/Enter repetido
    setIsSubmitting(true);

    // Validación mínima
    if (!email.trim()) {
      alert('Email obligatorio');
      setIsSubmitting(false);
      return;
    }

    await fakeRequest();
    alert(`Enviado: ${email}`);
    setIsSubmitting(false);
  };

  return (
    <section style={styles.section}>
      <h2>3.2. Evitar doble envío</h2>
      {/* fieldset disabled deshabilita todos los controles internos */}
      <fieldset disabled={isSubmitting} style={{ border: 'none', padding: 0 }}>
```

```
        <form onSubmit={handleSubmit}>
          <label style={styles.label}>
            Email
            <input
              type="email"
              value={email}
              onChange={(e)                          =>
setEmail(e.target.value)}
              placeholder="tu@correo.com"
              style={styles.input}
            />
          </label>
          <button                          type="submit"
disabled={isSubmitting}
style={styles.buttonPrimary}>
            {isSubmitting ? 'Enviando…' : 'Enviar'}
          </button>
        </form>
      </fieldset>
      <p    style={styles.tip}>Tip:    usa    un    flag
<code>isSubmitting</code>    y    deshabilita    el
botón      (y      opcionalmente      todo      el
<code>fieldset</code>).</p>
    </section>
  );
}

// === 3.3 Enviar un formulario desde código
===
function SubmitFromCodeDemo() {
  const formRef = useRef(null);
  const [query, setQuery] = useState('React');

  const handleSubmit = (ev) => {
    ev.preventDefault();
    alert(`Consulta enviada: ${query}`);
  };

  const submitProgrammatically = () => {
    const form = formRef.current;
    if (!form) return;
    // requestSubmit dispara el evento de submit y
respeta validaciones HTML
    if (typeof form.requestSubmit === 'function') {
      form.requestSubmit();
    } else {
      // Fallback: lanzar evento 'submit' manual
      form.dispatchEvent(new    Event('submit',    {
```

```
bubbles: true, cancelable: true }));
    }
  };

  return (
    <section style={styles.section}>
      <h2>3.3. Enviar desde código</h2>
      <form                                ref={formRef}
onSubmit={handleSubmit}>
        <label style={styles.label}>
          Búsqueda
          <input
            type="text"
            value={query}
            onChange={(e)                          =>
setQuery(e.target.value)}
            placeholder="¿Qué buscas?"
            style={styles.input}
            required
          />
        </label>
        <button                            type="submit"
style={styles.buttonPrimary}>Enviar</button>
      </form>
      <div style={{ marginTop: 8 }}>
        <button   onClick={submitProgrammatically}
style={styles.buttonSecondary}>
          Enviar formulario desde código
        </button>
      </div>
      <p                          style={styles.tip}>Tip:
<code>form.requestSubmit()</code> es la forma
moderna de disparar <code>onSubmit</code>
programáticamente.</p>
    </section>
  );
}

// === Estilos básicos ===
const styles = {
  app: { fontFamily: 'system-ui, -apple-system,
Segoe UI, Roboto, sans-serif', padding: 24,
lineHeight: 1.5 },
  section: { marginBottom: 24, padding: 16,
border: '1px solid #e5e7eb', borderRadius: 8,
background: '#fff' },
  label: { display: 'block', marginBottom: 8 },
  input: { display: 'block', width: '100%',
```

```
marginTop: 6, padding: '8px 10px', border: '1px solid #d1d5db', borderRadius: 6 },
  error: { marginTop: 6, color: '#dc2626', fontSize: 13 },
  info: { marginTop: 8, fontSize: 14, color: '#374151' },
  tip: { marginTop: 8, fontSize: 12, color: '#6b7280' },
  fieldRow: { marginBottom: 12 },
  buttonPrimary: { marginTop: 8, padding: '8px 12px', background: '#2563eb', color: '#fff', border: 'none', borderRadius: 6, cursor: 'pointer' },
  buttonSecondary: { padding: '8px 12px', background: '#111827', color: '#fff', border: 'none', borderRadius: 6, cursor: 'pointer' },
};

export default function App() {
  return (
    <div style={styles.app}>
      <header style={{ marginBottom: 16 }}>
        <h1>3. Validación de formularios: evento onSubmit</h1>
        <p>Ejemplos claros de validación, prevención de doble envío y submit programático.</p>
      </header>

      <ValidarFormulario />
      <DoubleSubmitDemo />
      <SubmitFromCodeDemo />

      <footer style={{ marginTop: 24, fontSize: 12, color: '#6b7280' }}>
        <p>Todos los formularios son controlados por estado. Usa onSubmit para centralizar la lógica.</p>
      </footer>
    </div>
  );
}
```

Aquí, renders.current se actualiza sin afectar al render, actuando como una variable estática interna al componente.

3. Validación básica

A. Controlled components

En React, un controlled component es un elemento de formulario cuyo valor está completamente gestionado por el estado de React.
Esto significa:

- El valor del input no vive dentro del DOM, sino en el estado del componente.
- Cada vez que el usuario escribe, React actualiza el estado.
- El input muestra siempre el valor que hay en el estado.
- React tiene el control total del dato → por eso se llama controlled.

Este patrón es fundamental porque:

- Permite validar en tiempo real.
- Permite transformar datos antes de mostrarlos.
- Permite sincronizar varios inputs entre sí.
- Permite enviar datos limpios y controlados.

Ciclo de funcionamiento de un Controlled Component

1. El usuario escribe en el *input*.
2. Se dispara *onChange*.
3. actualiza el estado con *setState*.
4. El componente se renderiza de nuevo.
5. El input muestra el valor actualizado del estado.

Es un bucle controlado por React, no por el navegador.

Ejemplo: Formulario con dos inputs controlados

/src/App.js

```
// src/Apps.js

import { useState } from "react";

export default function ControlledForm() {
  const [form, setForm] = useState({
    name: "",
    age: "",
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
```

```
    setForm({
      ...form,
      [name]: value, // actualiza solo el campo
modificado
    });
  };

  return (
    <form>
      <div>
        <label>Nombre</label>
        <input
          type="text"
          name="name"
          value={form.name}
          onChange={handleChange}
        />
      </div>

      <div>
        <label>Edad</label>
        <input
          type="number"
          name="age"
          value={form.age}
          onChange={handleChange}
        />
      </div>

      <p>Estado                                actual:
{JSON.stringify(form)}</p>
    </form>
  );
}
```

B. Validación manual

La validación manual significa que tú decides:

- Qué reglas debe cumplir cada campo
- Cuando validar (onChange, onBlur, onSubmit)
- Cómo mostrar los errores
- Qué hacer si el formulario no es válido

No dependes de librerías externas (Formik, Yup, RHF), sino que controlas todo con estado + funciones propias.

Ejemplo: Formulario con validación manual completa

/src/App.js

```
// src/Apps.js

import { useState } from "react";

export default function ManualValidationForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
    age: "",
  });

  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};

    // Validación del nombre
    if (!form.name.trim()) {
      newErrors.name = "El nombre es obligatorio";
    } else if (form.name.length < 3) {
      newErrors.name = "El nombre debe tener al
menos 3 caracteres";
    }

    // Validación del email
    if (!form.email.trim()) {
      newErrors.email = "El email es obligatorio";
    }     else     if     (!/^[\w-.]+@([\w-]+\.)+[\w-
]{2,4}$/.test(form.email)) {
      newErrors.email = "El email no es válido";
    }

    // Validación de la edad
    if (!form.age.trim()) {
      newErrors.age = "La edad es obligatoria";
    } else if (isNaN(form.age)) {
      newErrors.age = "La edad debe ser un
número";
    } else if (Number(form.age) < 18) {
      newErrors.age = "Debes ser mayor de edad";
    }
```

```
    setErrors(newErrors);

    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = (e) => {
    e.preventDefault();

    if (validate()) {
      alert("Formulario válido. Enviando datos...");
      // Aquí iría la lógica de envío real
    }
  };

  const handleChange = (e) => {
    setForm({
      ...form,
      [e.target.name]: e.target.value,
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Validación manual</h2>

      <div>
        <label>Nombre</label>
        <input
          type="text"
          name="name"
          value={form.name}
          onChange={handleChange}
        />
        {errors.name && <p style={{color:
"red"}}>{errors.name}</p>}
      </div>

      <div>
        <label>Email</label>
        <input
          type="email"
          name="email"
          value={form.email}
          onChange={handleChange}
        />
        {errors.email && <p style={{color:
"red"}}>{errors.email}</p>}
      </div>
```

```
      <div>
        <label>Edad</label>
        <input
          type="text"
          name="age"
          value={form.age}
          onChange={handleChange}
        />
        {errors.age && <p style={{color:
"red"}}>{errors.age}</p>}
      </div>

      <button type="submit">Enviar</button>
    </form>
  );
}
```

C. Gestión de errores

La gestión de errores en formularios consiste en:

- Detectar qué campos tienen errores
- Guardar esos errores en el estado
- Mostrar mensajes de error de forma clara
- Actualizar los errores cuando el usuario corrige los datos
- Limpiar errores cuando el formulario es válido

Todo esto se hace manualmente usando *useState*.

Cómo se gestionan los errores

1. **Crear un estado para los errores**: El estado suele ser un objeto donde cada propiedad representa un campo con error.

2. **Guardar errores dentro de la función de validación**: La función de validación revisa los campos y construye un objeto de errores.

3. **Mostrar los errores en el JSX**: Se suele mostrar debajo del input correspondiente.

Ejemplo: Gestión de errores completa

/src/App.js

```
// src/Apps.js

import { useState } from "react";

export default function ErrorManagementForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
  });

  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};

    // Validación del nombre
    if (!form.name.trim()) {
      newErrors.name = "El nombre es obligatorio";
    } else if (form.name.length < 3) {
      newErrors.name = "Debe tener al menos 3
caracteres";
    }

    // Validación del email
    if (!form.email.trim()) {
      newErrors.email = "El email es obligatorio";
    } else if (!/^[\w-.]+@([\w-]+\.)+[\w-
]{2,4}$/.test(form.email)) {
      newErrors.email = "El email no es válido";
    }

    setErrors(newErrors);

    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = (e) => {
    e.preventDefault();

    if (validate()) {
      alert("Formulario válido. Enviando datos...");
    }
  };

  const handleChange = (e) => {
```

```
    const { name, value } = e.target;

    setForm({
      ...form,
      [name]: value,
    });

    // Limpia el error del campo modificado
    setErrors({
      ...errors,
      [name]: "",
    });
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Gestión de errores</h2>

      <div>
        <label>Nombre</label>
        <input
          type="text"
          name="name"
          value={form.name}
          onChange={handleChange}
        />
        {errors.name && <p style={{color:
"red"}}>{errors.name}</p>}
      </div>

      <div>
        <label>Email</label>
        <input
          type="email"
          name="email"
          value={form.email}
          onChange={handleChange}
        />
        {errors.email && <p style={{color:
"red"}}>{errors.email}</p>}
      </div>

      <button type="submit">Enviar</button>
    </form>
  );
}
```